METHODS :-

* Action / Behaviours (Tasks performed by an object) cannot be stored using containers.

=> Method is a block of instructions which are used to perform a task

=> A method gets executed only when they are called.

▶ SYNTAX TO CREATE A METHOD :-

```
[Modifiers]  returntype  MethodName [(FormalArgum-ents)]
 (optional)                          (optional)
{
    // Java statements;
}
```

EXAMPLE :-

```
public static void add ( )
{
 Modifiers  return type  method name.
  int a = 10;
  int b = 20;
  int res = a+b;
  System.out.println(res)
}
```

METHOD TERMINOLOGIES:

1. METHOD SIGNATURE
   - a. Name of the Method
   - b. Formal arguments.
2. METHOD DECLARATION.
   a. Method signature
   b. Return type
   c. Modifiers
3. METHOD DEFINITION.
   a. Method declaration
   b. Method body / Implementation of method.

will continue

25/01/2022

▶ TYPES OF METHODS:-

Based on formal arguments, methods can be classified into two types:

1. No-Argument Method.
2. Parameterized Method.

▶ NO-ARGUMENT METHOD:-

A method which doesn't have a formal argument declared in it is known as No-argument Method.

Ex:
```
public static void main ()
{
 int a = 10;
 int b=20;
 Int res = a+b;
 System.out.println(res);
}
```

SYNTAX TO CALL NO ARGUMENT METHOD:

```
MethodName ();
```

How to provide authority to pass values to a method:

PARAMETERIZED METHODS:-

A Method which is having formal arguments declared in it is known as Parameterised method.

Ex:
```
public static void add (int a, int b)
{                          └→ Formal arguments.
 int res = a+b;
 System.out.println (res);
}
```

SYNTAX TO CALL PARAMETERIZED METHOD:

```
Method Name (actual. ~~formal~~ arguments);
```

Ex: ~~Method Name~~ add. (a, b);

Ex:
```
class M4
{
  // to create a parameterized method to perform addition of two numbers.
  // to perform a task for multiple times but for different inputs.
```

~~public static void main (String[] args)~~

{

~~System.out.println ("From main")~~

public static void add (int a, int b)

↳ Formal arguments

{

int res = a+b;

System.out.println (res);

}

public static void main (String [] args)

{

System.out.println ("From main");

add (10,20); // Method call statement

add (30,40); // Method call statement.

↳ Actual arguments.

}

}

FORMAL ARGUMENTS :

Formal Arguments are the containers (variables) created during the method declaration.

They are used to accept the values during the method call.

ACTUAL ARGUMENTS :

The values passed during the method call statement are known as Actual arguments

NOTE : Different Memory Areas :

1) Class Static Area : to store static members of class

2) Method Area : to store Methods.

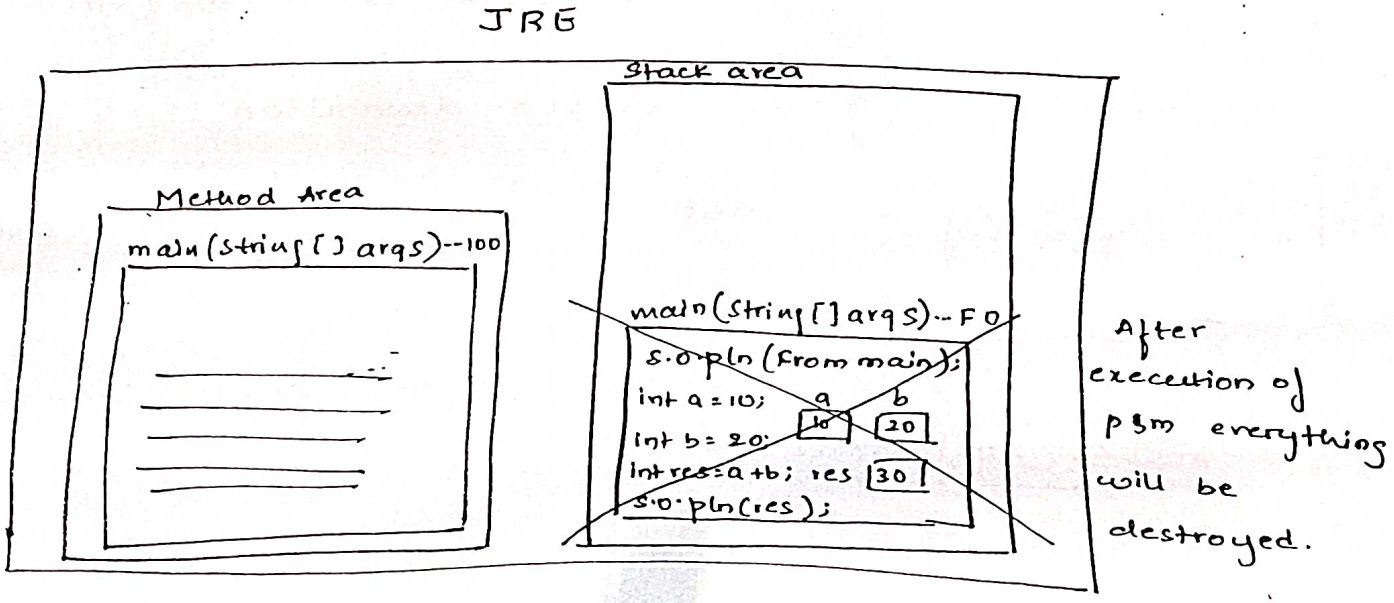
3) Stack Area : to execution purpose

4) Heap Area : to store non-static members of class

26/10/2022

Eg. class MS

```
{
  public static void main (String [] args)
  {
    System.out.println ('From Main');
    int a=10;
    int b=20;
    int res=a+b;
    System.out.println (res);
  }
}
```



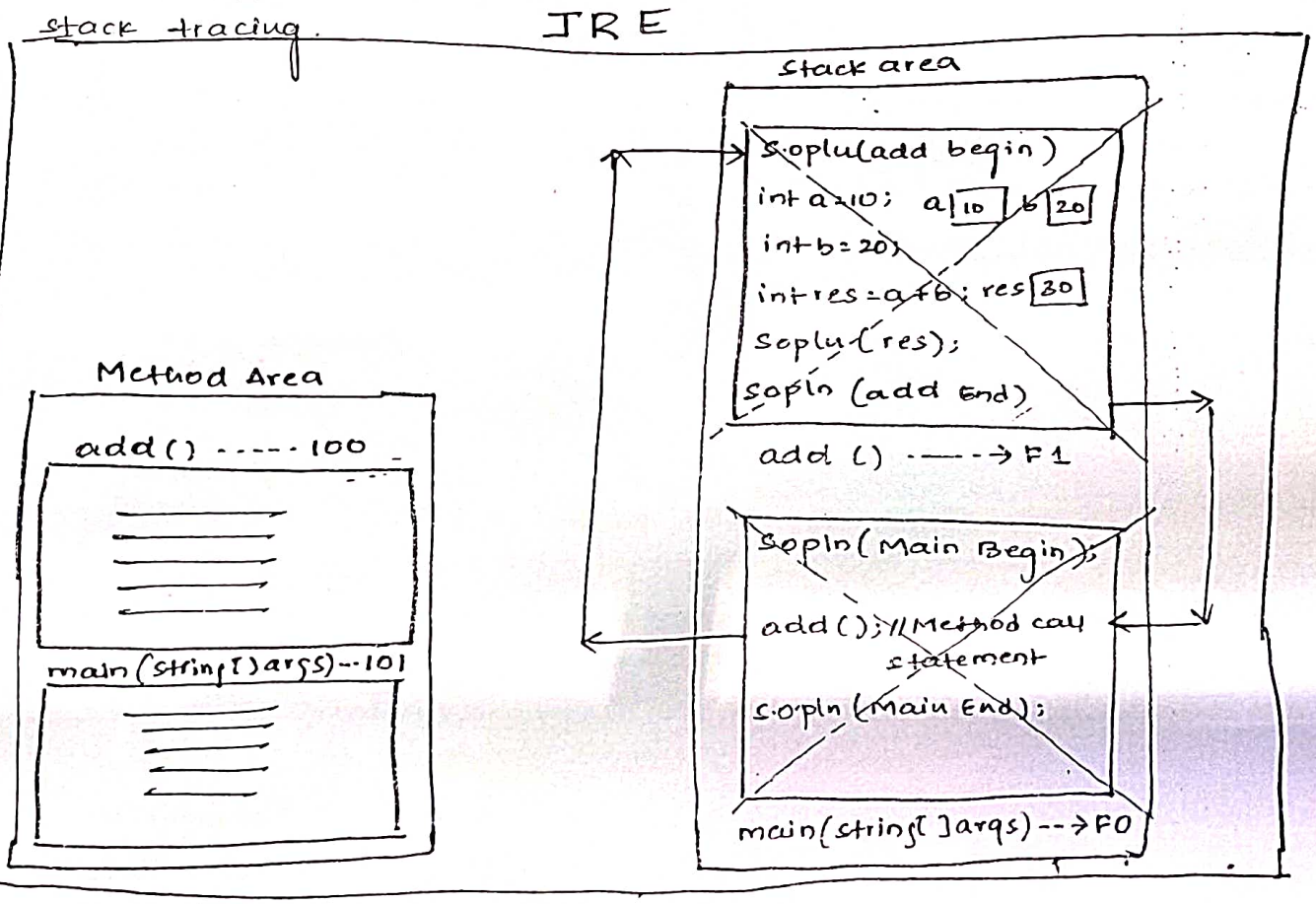
What happens when a method is called?

1. The execution of calling method is passed.
2. A new frame will be created for executing the called method.
3. The control will be transfered from calling method to the called method.
4. The called method will be under execution.
5. After the execution of called method the frame will be destroyed & control will be transfered back to calling method.
6. The execution of calling method resumes.

Eg:1.

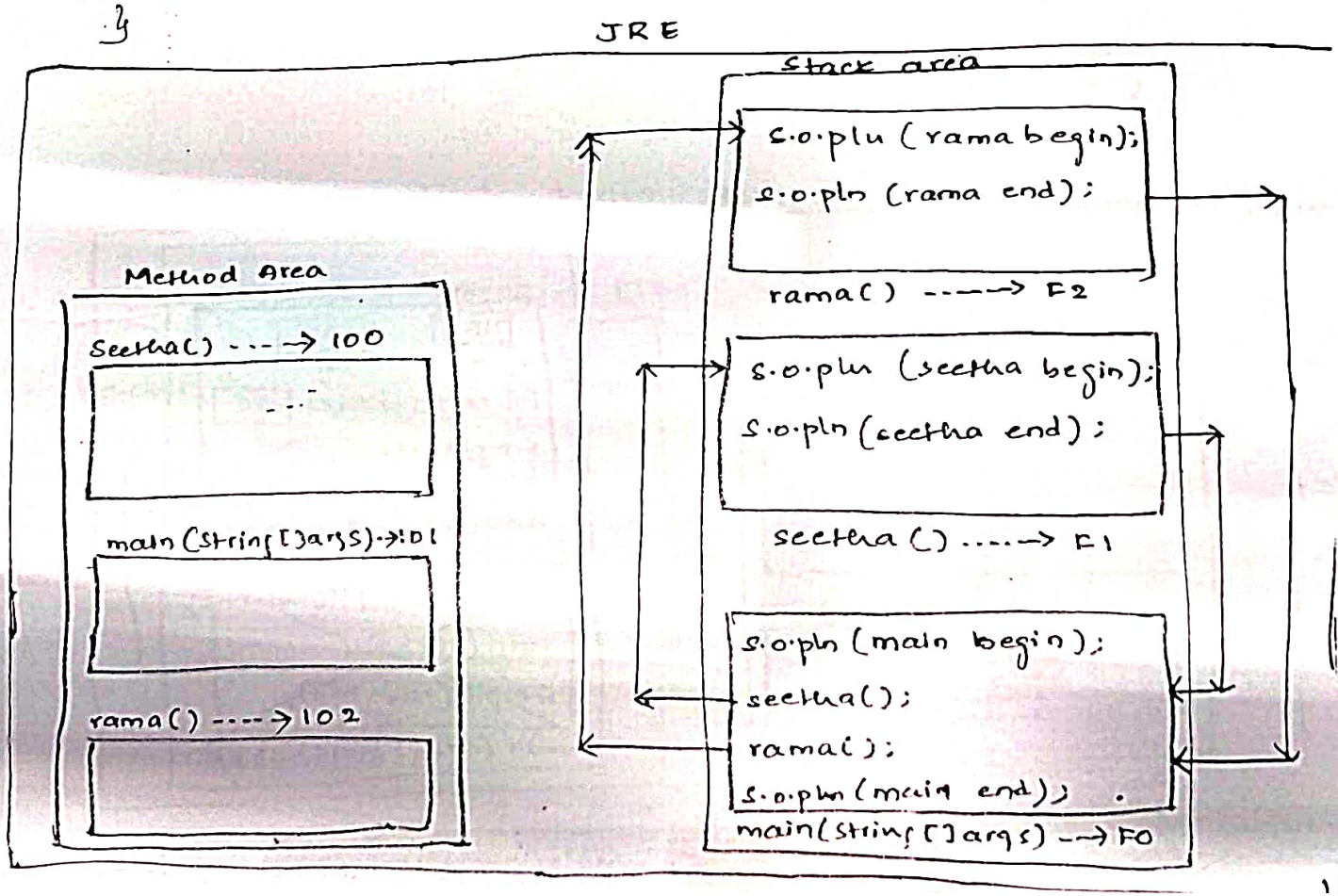
```
class M6
{
  //to perform addition of two numbers
  public static void add()
  {
  System.out.println("Add begin");
  int a=10;
  int b=20;
  int res= a+b;
  System.out.println(res);
  System.out.println("Add End");
  }
  public static void main(String[] args)
  {
  System.out.println("Main Begin");
  add()
  System.out.println("Main End");
  }
}
```

This is called as stack tracing.



Eg 2: class M7

```
{
public static void seetha()
{
System.out.println("seetha begin");
System.out.println("seetha end");
}
public static void main(String[] args)
{
System.out.println("Main begin");
seetha();
rama();
System.out.println("Main end");
}
public static void rama()
{
System.out.println("rama begin");
System.out.println("rama end");
}
}
```

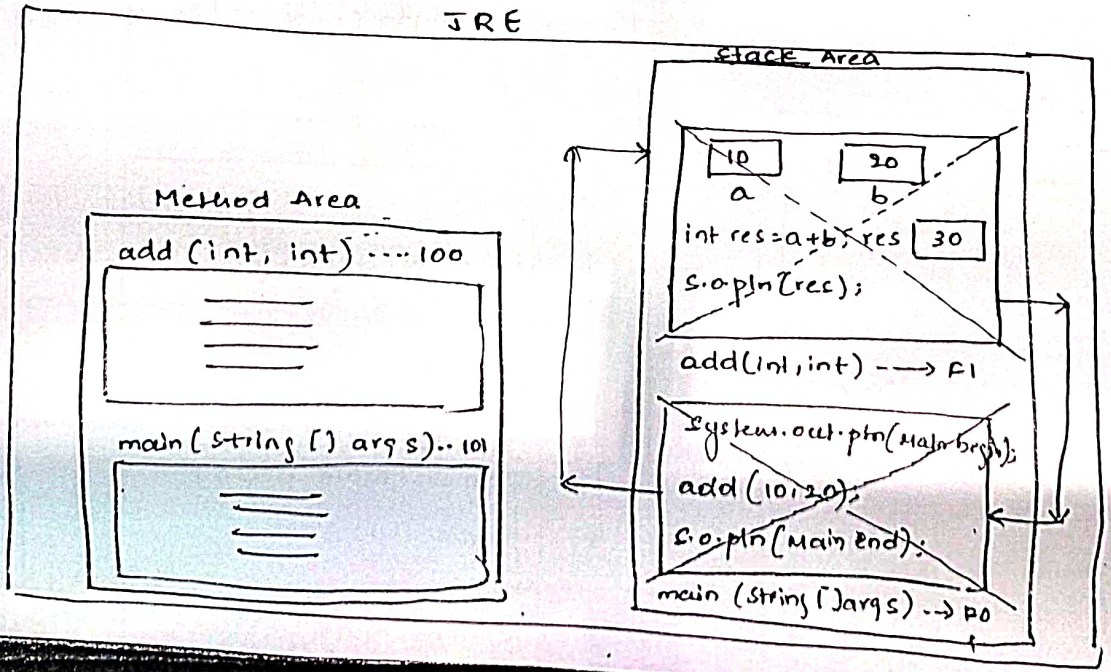


27/01/2022

Q. Can a user pass the i/p's to the method ?

Yes it is possible only when we have parameterised methods created. ie, whenever we have methods with formal arguements declared only then we can pass values.

Eg:
```
class M8
{
 public static void add(int a, int b)
 {
  int res=a+b;
  System.out.println(res);
 }
 public static void main(String[] args)
 {
 System.out.println("Main begin");
 add(10,20);
 System.out.println("Main End");
 }
}
```



O/p:

Main begin

30

Main End

NOTE: The formal arguments declared in a method will always be a local variable to that method.

ie. formal arguments declared in another method cannot be used in ~~meth~~ main method.

2. The compiler always tries to widen the actual arguments, when ever we pass different type value. If it is not possible only ~~when~~ then we get CTE

RULES TO CALL A METHOD :-

1. The number of formal arguments and actual arguments must be same.

2. The corresponding types of actual & formal argume must also be same. Else compiler tries for widening. If it is not possible then it throws CTE.

Q. Design a method to find largest of two numbers.

```
class largestNum()
{
  public static void large_num(int a, int b)
  {
  ~~int res;~~
  if(a>b)
     system.out.println(a);
  else
     System.out.println(b);
  }
  public static void main(String[] args)
  {
  System.out.println("The largest of two numbers is:")
  large_num(10,20);
  }
}
```

Q. Design a method to check whether the numbe is even or odd.

```
class Number()
{
 public static void num(int a)
 {
 if (a%2==0)
    System.out.println(a+" is Even number");
 else
    System.out.println(a+" is Odd number");
 }
 public static void main(String[] args)
 {
 System.out.println("The given number:");
 num(35);
 }
}
```

Q. Design a method to print numbers from 1 to 10

```
class PrintNum()
{
public static void print(int a, int b)
{
 for(int i=a ; i<=b ; i++)
  {
    System.out.println(i);
  }
}
public static void main(String[] args)
{
print.(1,10);
}
}
```

~~Q. Design a meth~~

Q. Design a method to check whether the numb is prime or not.

```
class PrimeNumber()
{
public static void prime(int num)
{ int temp=2;
 if (num!=1)
 {
  while (temp<num)
   {
     if (num%temp==0)
      {
        System.out.println("num+ is not a prime number");
        break;
      }
      temp++;
   }
  else
     System.out.println("num+ is a prime number");
 }
else
     System.out.println("num+ is not a prime number");
}
public static void main (String[] args)
{
System.out.print("The given number ");
prime(5);
}
}
```

'return' keyword :

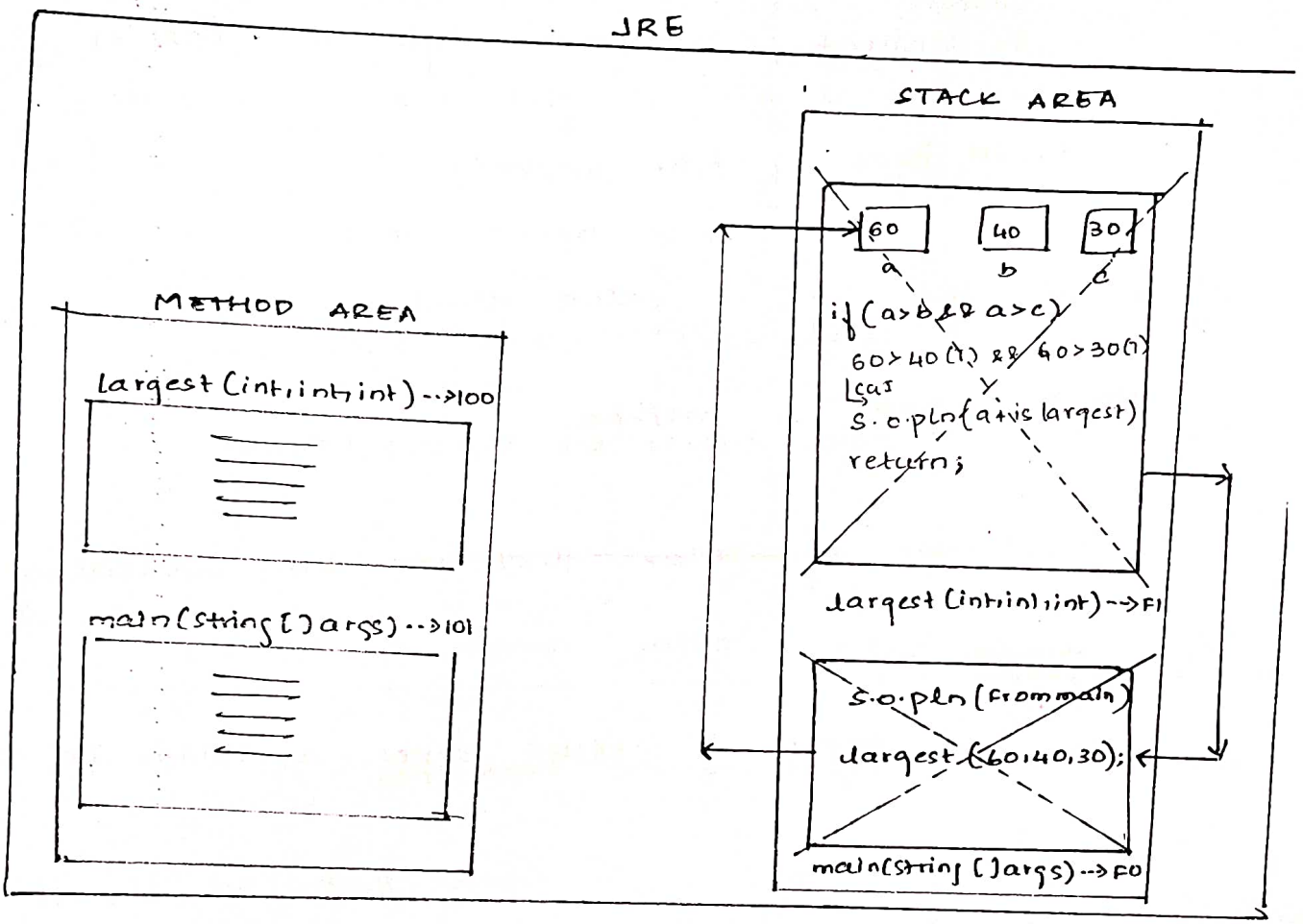
* return is a keyword.
* It destroys the frame where it is used and transfers the control back to the calling method.
* Therefore, it has to be used as last instruction of the block, else we get CTE.

Ex:
```
public static void sheela()
{ ...
System.out.println("sheela Begin");
return; // CTE bcz of Unreachable statement
System.out.println("sheela End"); // Unreachable statement
}
```

Ex:

```
class LargestNum
{
public static void largest(int a, int b, int c)
{
 if(a>b && a>c)
 {
    System.out.println(a + "is largest");
  return;
 }
if(b>a && b>c)
 {
   System.out.println(b + "is largest");
   return;
 }
if (c>a && c>b)
 {
   System.out.println(c + "is Largest");
   return;
 }
}
public static void main(String[] args)
{
   System.out.println("From main");
   largest(60, 40, 30);
 }
}
```

Stack tracing:



O/P:

From main

60 is largest

return type:-

Can a method return a value to its caller?

-> Yes, we can return the following types of data/ literals:-

1. Numericals
2. Characters
3. String.
4. Boolean

If I am returning a value, I have to provide its datatype & that type we call it as return type of the method.

defn: It is the type of value that we are returning from a method.

~~The types of value that we are returning from a method~~

The types of return types available in java are:

1. void
2. primitive data types.
3. non-primitive data types.

void: void means nothing. We can use void as return type to methods which doesn't return anything to its caller.

If the return type of method is void then it is not mandatory to return a value, if we want we can have just return statement

but we cannot return a value from this method. If we try to do it, we get CTE.

Ex: public static void test()
{
System.out.println("from test()");
return 10; // CTE bz return type is void.
}

To design a method which returns a value:-

If we need to design a method which returns a value, then it is mandatory to take the return type of the method anything other than void. ie, either primitive type or non-primitive type.

Whenever the return type is anything other than void, it becomes mandatory to return a value from that method to its caller, else we get CTE

Ex1: public static int print()
{
System.out.println("from print()"); // CTE as there is no return statement
}

Ex: 2 :

```
public static int print()
{
System.out.println ("From print()");
return; // CTE as there is no return value
}
```

Ex 3 :

```
Public static int print()
{
System.out.println ("from print()");
return 10; // CTS
}
```

Questions :

1. Design a method which returns summation of two numbers.
2. Design a method which returns largest of 2 nos
3. Design a method which returns factorial of a no.
4. Design a method which returns ASCII value of a character.

=> Design a method which returns largest of 2 no.

```
class largest
{
 public static int large (int a, int b)
 {
  if (a> b)
     {
      return a;
      }
  else
     {
      return b;
     }
 }
 public static void main (string [] args)
  {
  System.out.println(" The largest of two numbers is : ");
  System.out.println (large (20, 10));
  }
}
```

3) Design a method which returns factorial of a number.

```
Class Factorial
{
 public static int fact (int a)
 {
  int res = 1;
  for (int temp = a; temp > 1; temp--)
    {
      res *= temp;
    }
   return res;
  }
  public static void main (String [] args)
  {
   int num = 5;
   System.out.println (" The factorial of "+ num+ "is :"+fact(n
   }
}
```

4) Design a method which returns ASCII value of a character.

```
class ASCII
{
 public static int ascii(char ch)
  {
  return ch;
  }
  public static void main(String[] args)
  {
   char ch = '#';
   int asc = ascii(ch);
   System.out.println("The ASCII value of "+ch+" is :"+asc);
  }
}
```

o/p

for A ⇒ 65

⇒ 35.